

Plano de Implementação — Gestão de Materiais

Gestão de Materiais Corporativo

Gerado em 26/09/2026

Fonte: IMPLEMENTATION-PLAN.md

Objetivo

Evoluir a fundação publicada para uma plataforma corporativa completa, usando entregas verticais, TDD, CI e commits pequenos.

Estado atual

Concluído e publicado:

- Sprint 01 — fundação técnica;
- Sprint 02 — identidade e autorização inicial;
- Sprint 03 — catálogo de materiais;
- Sprint 04 — estoque, reservas, transferências e API;
- Sprint 05 — solicitações, aprovação e API.

Validação atual:

```
27 testes backend passando
Django check sem erros
Next lint passando
Next build passando
```

Ordem de execução

Fase 1 — Hardening da base

1. Configurar PostgreSQL por DATABASE_URL;
2. Separar settings dev, homologação e produção;
3. Configurar CORS por ambiente;
4. Adicionar health check;
5. Adicionar documentação OpenAPI;
6. Configurar rate limit de autenticação;
7. Criar auditoria de login e mutações;
8. Configurar CI com cache e cobertura.

Aceite: ambiente limpo executa migrations, testes e check sem dependência de SQLite de produção.

Fase 2 — Sprint 06: compras

1. Criar testes de fornecedor;
2. Criar testes de cotação;
3. Criar testes de pedido;
4. Implementar apps/purchasing;
5. Implementar fornecedor e contato;
6. Implementar solicitação de compra;
7. Implementar cotação e itens;
8. Implementar comparação;

9. Implementar pedido;
10. Implementar recebimento parcial;
11. Vincular recebimento a movimentação de entrada;
12. Validar nota fiscal e divergências;
13. Criar API;
14. Criar telas Next;
15. Rodar suíte completa;
16. Commitar e publicar.

Aceite: pedido recebido gera entrada idempotente no estoque e divergências ficam registradas.

Fase 3 — Sprint 07: entrega e custódia

1. Criar modelo de ativo individual;
2. Criar custódia;
3. Criar termo;
4. Criar assinatura;
5. Criar entrega;
6. Integrar entrega com reserva e saída;
7. Criar devolução;
8. Criar troca;
9. Criar manutenção;
10. Criar pendências de desligamento;
11. Criar QR Code;
12. Criar API e telas;
13. Testar dupla custódia;
14. Publicar.

Aceite: um ativo não pode possuir duas custódias ativas e toda entrega tem evidência.

Fase 4 — Sprint 08: inventário e auditoria

1. Criar inventário por local;
2. Criar contagem;
3. Criar divergência;
4. Criar aprovação de ajuste;
5. Criar auditoria imutável;
6. Criar exportação de evidências;
7. Testar inventário parcial;
8. Publicar.

Aceite: diferença de inventário gera ajuste rastreável e não altera o histórico anterior.

Fase 5 — Sprint 09: dashboards

1. Definir consultas e índices;
2. Criar endpoints agregados;
3. Criar dashboard global;
4. Criar dashboard por área;
5. Criar relatórios de estoque;
6. Criar relatório de consumo;
7. Criar relatório de compras;
8. Criar exportações;
9. Testar filtros e escopo;
10. Publicar.

Aceite: nenhum indicador exibe dados fora do escopo do usuário.

Fase 6 — Sprint 10: IA

1. Definir contratos do ai-service;
2. Criar autenticação serviço a serviço;
3. Criar fila de jobs;
4. Criar classificação de solicitações;
5. Criar previsão de consumo;
6. Criar OCR de notas;
7. Criar busca semântica;
8. Criar explicação e confiança da recomendação;
9. Exigir aprovação humana para efeitos críticos;
10. Publicar com feature flags.

Aceite: IA falhando não bloqueia operações transacionais e nenhuma ação crítica ocorre sem autorização humana.

Fase 7 — Sprint 11: produção

1. Criar ambientes;
2. Configurar deploy backend;
3. Configurar deploy frontend;
4. Configurar migrations seguras;
5. Configurar backup;
6. Testar restauração;
7. Configurar monitoramento;
8. Configurar alertas;
9. Revisar LGPD;

- 10. Fazer teste de carga;
- 11. Executar checklist de go-live.

Aceite: produção possui rollback, backup restaurável, observabilidade e documentação operacional.

TDD por tarefa

Para cada regra:

```
RED    → escrever teste que falha
GREEN  → implementar o mínimo
REFACTOR → simplificar sem quebrar
VERIFY → suíte completa e check
COMMIT → commit convencional
PUSH   → branch e main atualizadas conforme política
```

Convenção de commits

```
feat(scope): nova funcionalidade
fix(scope): correção
refactor(scope): refatoração
test(scope): testes
docs(scope): documentação
ci(scope): pipeline
```

Gates obrigatórios

Backend:

```
cd backend
.venv/Scripts/python.exe -m pytest -q
.venv/Scripts/python.exe manage.py check
```

Frontend:

```
cd frontend
npm run lint
npm run build
```

Git:

```
git diff --check
git status --short --branch
```

Riscos

Risco	Mitigação
Crescimento prematuro em microsserviços	Monólito modular até necessidade real
IA comprometer fluxo crítico	Jobs assíncronos e aprovação humana
Saldo inconsistente	Transação, lock, constraints e idempotência
Acesso entre áreas	Escopo validado no backend
Upload inseguro	Validação, armazenamento externo e limites
Deploy quebrar migration	Pipeline com migration explícita e backup

Risco	Mitigação
Dashboard lento	Consultas agregadas, índices e cache

Definição de pronto

Uma sprint só termina quando:

- Código está implementado;
- Testes cobrem regras críticas;
- Teste falhou antes da implementação;
- Suíte completa passa;
- Documentação foi atualizada;
- CI está verde;
- Commit foi publicado;
- Não existem efeitos externos não verificados.